Resgate em Labirinto Dinâmico com Chaves e Portas

Mickaelly Freitas Nobre[1]

[1]ICMC – Departamento de Pós-Graduação – Universidade de São Paulo - São Carlos
Inteligência Artificial
CEP 13566-590 – São Carlos – SP – Brasil

mickaellyfreitas07@usp.br

Abstract. *This work addresses the problem of autonomous rescue in a dynamic maze with locked doors and keys, in which a robot must collect specific keys to unlock doors and reach a patient. The environment is static, deterministic, and fully observable, with cumulative key collection. Two search algorithms are implemented and compared: BFS and A* search with a Manhattan distance heuristic. Experiments are conducted across six distinct maze scenarios, varying in complexity, obstacles, and dependencies between keys and doors.*

Resumo. *Este trabalho aborda o problema de resgate autônomo em um labirinto dinâmico com portas trancadas e chaves, no qual um robô deve coletar chaves específicas para destrancar portas e alcançar um paciente. O ambiente é estático, determinístico e totalmente observável, com coleta acumulativa de chaves. Dois algoritmos de busca são implementados e comparados:BFS e A* com heurística de Manhattan. Os experimentos são realizados em seis cenários distintos de labirinto, variando em complexidade, obstáculos e dependências entre chaves e portas.*

1. Introdução

Um robô autônomo de resgate deve localizar e alcançar um paciente preso em um labirinto contendo portas trancadas que só podem ser abertas por meio de chaves específicas distribuídas pelo ambiente. No labirinto, diferentes tipos de células representam os elementos do problema: o caminho livre é indicado por `.`, permitindo a movimentação do robô, enquanto as paredes (`#`) bloqueiam completamente a passagem. A posição inicial do robô é representada por `R` e o objetivo, correspondente ao paciente, por `P`, ambos únicos no cenário. O robô pode se mover em quatro direções (`CIMA`, `BAIXO`, `ESQUERDA` e `DIREITA`), sem a possibilidade de atravessar paredes. As chaves (`K_A`, `K_B`, etc.) são coletadas automaticamente ao serem alcançadas, passando a integrar o conjunto de chaves do robô, enquanto as portas correspondentes (`D_A`, `D_B`, etc.) somente podem ser atravessadas caso a chave adequada já tenha sido obtida. Caso o robô tente atravessar uma porta sem possuir a chave correspondente, o movimento é bloqueado. Além disso, as chaves não são consumidas após o uso, permanecendo disponíveis durante toda a execução. À medida que novas chaves são coletadas, regiões anteriormente inacessíveis tornam-se disponíveis, caracterizando um ambiente com dependência entre ações e expansão dinâmica do espaço de estados.

O objetivo do problema é encontrar a trajetória de menor custo, isto é, o menor número de movimentos necessários para conduzir o robô desde a posição inicial até o paciente, independentemente das chaves acumuladas ao longo do caminho. Esse problema é

relevante por representar um cenário realista de navegação com restrições, frequentemente encontrado em aplicações de robótica de resgate, exploração autônoma e planejamento inteligente. Além disso, o problema é adequado para o estudo e comparação de algoritmos de busca em Inteligência Artificial, permitindo analisar estratégias de busca cega e informada. Nesse contexto, foram escolhidos os algoritmos BFS (*Breadth-First Search*) e A* com heurística de *Manhattan*. A escolha foi estratégica por permitir o contraste entre busca cega e busca informada: enquanto o BFS explora o espaço de estados sem utilizar conhecimento adicional sobre o objetivo, garantindo a obtenção de uma solução ótima em problemas de custo uniforme, o A* utiliza heurísticas para direcionar a exploração do espaço de busca, reduzindo a quantidade de estados visitados e aumentando a eficiência computacional, especialmente em ambientes cujo espaço de estados cresce rapidamente devido à combinação entre posições possíveis e subconjuntos de chaves coletadas.

2. Divisão Estrutural

O algoritmo foi implementado em Python e desenvolvido no ambiente Google Colab, sendo organizado em seis módulos principais: Módulo 1 — Definição do Estado e do Labirinto; Módulo 2 — Algoritmo BFS; Módulo 3 — Algoritmo A* (com heurística de Manhattan); Módulo 4 — Gerador de Cenários; Módulo 5 — Analisador Comparativo de Resultados; e Módulo 6 — Função Principal.

2.1. Formalização do Problema

A formalização do problema está concentrada no Módulo 1, por meio das classes `Estado` e `Labirinto`, caracterizando um problema clássico de busca em espaço de estados na área de Inteligência Artificial. O problema pode ser representado pela tupla $P = (S, A, T, s_0, G, C, h)$, em que S representa o conjunto de estados, A o conjunto de ações possíveis, T a função de transição, s_0 o estado inicial, G o teste de objetivo, C a função de custo e h a função heurística utilizada pelo algoritmo de busca. Cada estado armazena a posição atual do robô (x, y), o conjunto de chaves coletadas, o caminho percorrido, a profundidade e o custo acumulado da busca. Formalmente, um estado pode ser descrito como $s = (x, y, K)$, em que K representa o conjunto de chaves obtidas pelo agente ao longo da exploração. A unicidade dos estados é garantida pelo método `hash`, baseado na posição do agente e no conjunto de chaves coletadas, permitindo o armazenamento eficiente de estados visitados e evitando expansões redundantes.

O espaço de estados é formado por todas as posições válidas do labirinto combinadas com os possíveis subconjuntos de chaves coletadas. O estado inicial corresponde à posição do robô identificada pelo símbolo `R`, com conjunto vazio de chaves, ou seja, o agente inicia a busca sem possuir nenhum item. O objetivo do problema é alcançar a posição do paciente representada pelo símbolo `P`, independentemente das chaves acumuladas durante a navegação. O conjunto de ações possíveis consiste nos movimentos `CIMA`, `BAIXO`, `ESQUERDA` e `DIREITA`, caracterizando um ambiente discreto baseado em movimentação ortogonal.

A função de transição, implementada pelo método `transicao`, define a dinâmica do sistema ao aplicar uma ação válida sobre um estado atual, produzindo um novo estado. Essa transição atualiza a posição do robô, coleta automaticamente chaves do tipo `K_X` quando presentes na célula acessada, incrementa a profundidade da busca e

atualiza o custo acumulado do caminho. As ações aplicáveis são restringidas pelos limites do labirinto, impedem a travessia de paredes representadas pelo símbolo # e permitem a passagem por portas `D_X` apenas quando a chave correspondente já foi coletada anteriormente. Dessa forma, o problema incorpora dependências condicionais entre estados, tornando a navegação mais complexa do que um simples problema de caminho mínimo em grade.

A função de custo considera custo unitário para cada movimento realizado, isto é, cada transição possui custo igual a 1, caracterizando um problema de custo homogêneo. Para guiar a busca, o algoritmo A* utiliza a heurística da distância de Manhattan, que estima o custo restante até o objetivo considerando apenas deslocamentos horizontais e verticais:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

Nessa expressão, (x_n, y_n) representa a posição atual do agente e (x_g, y_g) representa a posição do objetivo. Essa heurística é admissível para problemas de navegação em grade sem movimentos diagonais, pois não superestima o custo real até o destino. Por fim, o problema pode ser classificado como determinístico, totalmente observável, estático, discreto e monoagente, no qual o conjunto de chaves coletadas é acumulativo e não sofre perdas ao longo da execução.

2.2. Algoritmos

Os módulos 2 e 3 implementam algoritmos de busca em labirintos com coleta de chaves, utilizando estratégias distintas de exploração do espaço de estados. O Módulo 2 implementa a Busca em Largura (BFS), na qual os estados são explorados por níveis, expandindo todos os nós de uma mesma profundidade antes de avançar para a próxima. Para isso, a classe utiliza uma fila do tipo `deque` e um conjunto de visitados, garantindo a obtenção do caminho mínimo em número de movimentos.

Já o Módulo 3 implementa o algoritmo A* (A-estrela) com heurística de Manhattan, utilizando uma fila de prioridade baseada em `heapq`. Nesse método, os estados são expandidos de acordo com o menor custo estimado total, definido por

$$f(n) = g(n) + h(n)$$

em que $g(n)$ representa o custo real acumulado até o estado atual e $h(n)$ corresponde à estimativa de custo até o objetivo calculada pela heurística de Manhattan. Para o controle dos custos, o algoritmo utiliza os dicionários `g_score` e `f_score`, tornando a busca mais eficiente em labirintos maiores quando comparada ao BFS.

Ambos os algoritmos foram implementados como classes independentes, contendo métodos `__init__()` para armazenar o labirinto e inicializar estatísticas, além do método `buscar()`, responsável pela execução da busca e retorno do caminho encontrado juntamente com métricas de desempenho. Os dois módulos também incluem funções para reinicialização de estatísticas, exibição detalhada de resultados e visualização do caminho percorrido.

Além disso, ambos realizam avaliação completa de desempenho, incluindo número de nós gerados e expandidos, tempo de execução, pico de memória utilizando

`tracemalloc`, tamanho máximo da fronteira e métricas de qualidade da solução. Enquanto o BFS é mais adequado para garantir soluções ótimas em ambientes sem informação heurística, o A* apresenta maior eficiência ao direcionar a busca com conhecimento estimado do objetivo.

2.3. Configuração dos Experimentos

A configuração dos experimentos neste código é estruturada em três módulos principais, que definem como os testes são realizados.

O Módulo 4 (`GeradorCenarios`) utiliza essa estrutura para gerar seis cenários distintos de labirinto, permitindo a avaliação dos algoritmos BFS e A* em diferentes níveis de complexidade. Entre eles, há um cenário simples sem obstáculos; um com uma chave e uma porta, que exige coleta; um mais complexo com múltiplas chaves e portas, explorando dependências; um cenário sem solução, onde falta a chave necessária; um ambiente maior (15x15) com paredes aleatórias para análise de escalabilidade; e um cenário desafiador, no qual a chave está posicionada em um beco, exigindo maior eficiência heurística.

O Módulo 5 (`AnalisadorComparativo`) coleta métricas detalhadas, incluindo nós gerados, nós expandidos, tempo de execução (`time.time()`), pico de memória (`tracemalloc`) e tamanho máximo da fronteira, além de métricas qualitativas como sucesso, custo da solução (número de movimentos) e chaves coletadas; ele compara BFS e A* com tabelas, calcula reduções percentuais (nós, tempo e memória), verifica otimalidade, classifica a heurística (Excelente, Boa, Moderada ou Limitada) e analisa convergência e limitações.

Por fim, o Módulo 6 (função principal) orquestra a execução sequencial dos seis experimentos, realizando a exibição do labirinto, a execução do BFS (com fila `deque` no modelo FIFO e controle de estados visitados) e do A* (com heurística de Manhattan e fila de prioridade `heapq`), além da coleta das métricas de desempenho e seu encaminhamento ao analisador comparativo.

Ao final, o sistema pode gerar gráficos comparativos com `matplotlib` (nós expandidos, tempo, memória e redução percentual) e um relatório consolidado.

3. Resultados

O projeto está disponível no GitHub[1], contendo código-fonte e README com descrição, tabela comparativa e gráficos; os testes foram organizados em seis casos (um por labirinto), reportando métricas como qualidade e custo da solução, estados explorados, tempo de execução, uso de memória e caminho encontrado para cada algoritmo, além de uma pasta "Resultados" com imagens da tabela, gráficos (memória, estados explorados, tempo, nós expandidos). A Tabela 1 mostra a custo, o tempo de execução, os nós expandidos, tempo e memória de cada um dos algoritmos.

No primeiro labirinto, também chamado de simples (sem obstáculos), foi utilizado um labirinto simples, sem obstáculos, em que o robô inicia na posição R e deve alcançar o paciente P por caminhos livres, sem portas ou chaves, tornando o problema direto. O BFS encontrou solução ótima com 4 movimentos (todos para a direita, de (0,0) até

[1] `https://github.com/mickaelly01nobre/Inteligencia-Artificial`

Tabela 1. Tabela Comparativa entre BFS A*

Cenário	Alg.	Sol.	Custo	Nós Exp.	Tempo (s)	Mem. (MB)	Front. Máx
1. Simples (sem obstáculos)	BFS	✓	4	14	0.0008	0.01	4
	A*	✓	4	5 (36%)	0.0003 (40%)	0.00	5
2. Chave A e Porta A	BFS	✓	8	29	0.0012	0.02	6
	A*	✓	8	20 (69%)	0.0014 (118%)	0.01	5
3. Complexo (2 chaves/2 portas)	BFS	✓	9	50	0.0023	0.03	18
	A*	✓	9	26 (52%)	0.0015 (66%)	0.02	18
4. Sem solução (falta chave)	BFS	✓	8	18	0.0007	0.01	3
	A*	✓	8	11 (61%)	0.0005 (70%)	0.01	3
5. Grande (15x15)	BFS	✓	28	361	0.0217	0.23	29
	A*	✓	28	227 (63%)	0.0149 (69%)	0.15	31
6. Heurística desafiadora	BFS	✓	8	27	0.0012	0.01	5
	A*	✓	8	19 (70%)	0.0009 (79%)	0.01	4

(0,4)), gerando 17 nós, expandindo 14, com fator de ramificação médio de 1,21, tempo de execução de aproximadamente $0{,}0008$ segundos e uso de cerca de $0{,}01$ MB de memória no pico.

Em seguida, foi aplicado o algoritmo A* com a heurística de *Manhattan*. Assim como o BFS, o A* também encontrou uma solução ótima de 4 movimentos, seguindo exatamente o mesmo caminho. No entanto, apresentou maior eficiência, gerando apenas 9 nós e expandindo 5, apesar de ter um fator de ramificação médio um pouco maior (1,80). O tempo de execução foi menor, cerca de 0,0003 segundos, com uso de memória praticamente nulo. A Figura 1 apresenta a comparação do uso de memória entre os algoritmos em cada cenário de labirinto, desde o primeiro, caracterizado como um labirinto simples sem obstáculos, até o último, que representa um cenário com heurística desafiadora.

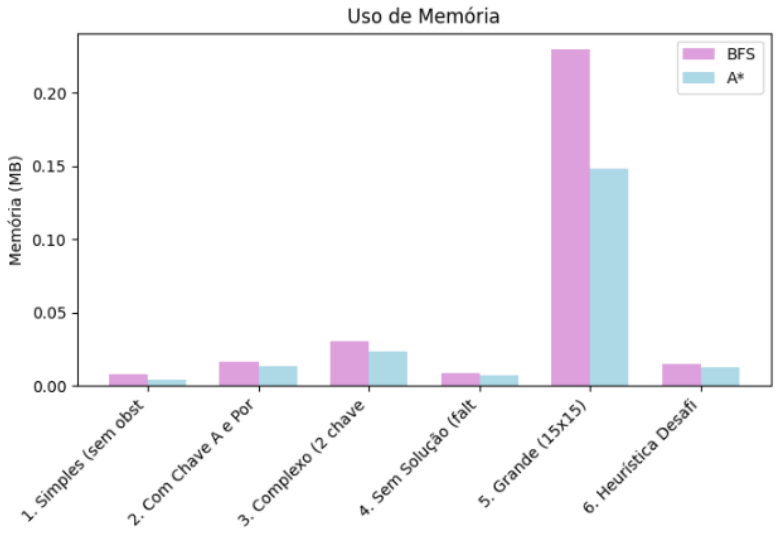

Figura 1. Gráfico do Uso da Memória

No segundo labirinto, também chamado de Chave A e Porta A, o labirinto é mais complexo por conter uma chave K_A, uma porta D_A e paredes que restringem o caminho, exigindo que o robô colete a chave antes de acessar a porta e chegar ao paciente. O BFS encontrou solução ótima com 8 movimentos, indo para baixo até a linha da chave, depois à direita para coletá-la e seguir até a porta e o objetivo, gerando 33 nós, expandindo 29, com fator de ramificação médio de 1,14, tempo de aproximadamente 0,0012 segundos e

uso de memória de cerca de 0,02 MB.

Já o algoritmo A*, utilizando a heurística de *Manhattan*, também encontrou a mesma solução ótima com os mesmos 8 movimentos e caminho idêntico ao BFS. No entanto, apresentou melhor desempenho, gerando 24 nós e expandindo 20, com fator de ramificação médio de 1,20. O tempo de execução foi ligeiramente menor, cerca de 0,0014 segundos, e o uso de memória foi de aproximadamente 0,01 MB. A Figura 2 apresenta a comparação do tempo de execução entre os algoritmos em cada cenário de labirinto, desde o primeiro, caracterizado como um labirinto simples sem obstáculos, até o último, que representa um cenário com heurística desafiadora.

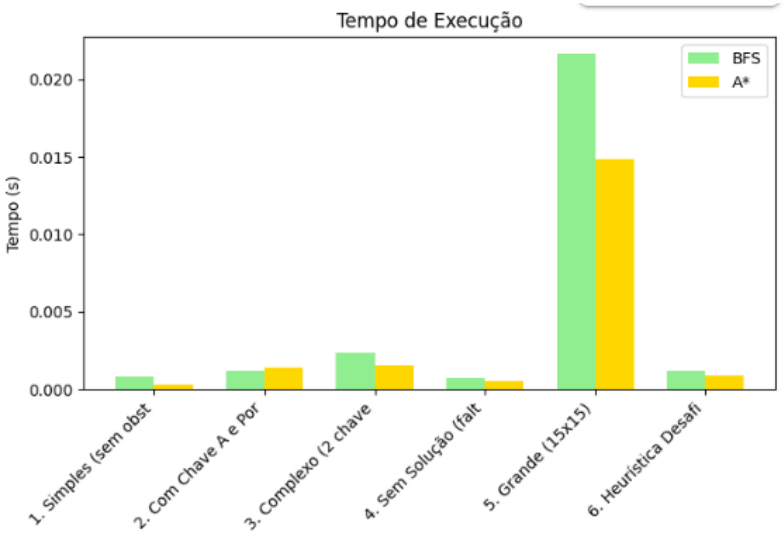

Figura 2. Gráfico do Tempo de Execução

No terceiro labirinto, também nomeado de complexo (2 chaves/ 2 portas), foi utilizado um labirinto mais complexo com duas chaves (A e B), duas portas correspondentes e paredes que dificultam a navegação, mas o caminho até o paciente não exigiu a coleta de chaves por existir uma rota livre contornando os obstáculos. O BFS encontrou solução ótima com 9 movimentos, descendo até a última linha e depois indo à direita até o objetivo, gerando 66 nós, expandindo 50, com fator de ramificação médio de 1,32, tempo de execução de aproximadamente 0,0023 segundos e uso de memória de cerca de 0,03 MB.

O A*, utilizando a heurística de *Manhattan*, também encontrou a mesma solução ótima do BFS com 9 movimentos e o mesmo caminho, porém apresentou melhor desempenho, gerando 43 nós e expandindo 26, com fator de ramificação médio de 1,65, tempo de execução de aproximadamente 0,0015 segundos e uso de memória de cerca de 0,02 MB. A Figura 3 apresenta a comparação de ganho entre os algoritmos em cada cenário de labirinto, desde o primeiro, caracterizado como um labirinto simples sem obstáculos, até o último, que representa um cenário com heurística desafiadora.

No quarto labirinto, também identificado de sem solução (falta chave), há uma porta D_A bloqueando o caminho até o paciente e nenhuma chave disponível, caracterizando um problema sem solução real; mesmo assim, tanto o BFS quanto o A* encontraram um caminho até o objetivo com 8 movimentos, contornando os obstáculos por áreas livres do mapa, com trajeto que consiste em descer até uma região aberta, seguir para a direita e depois subir até o objetivo.

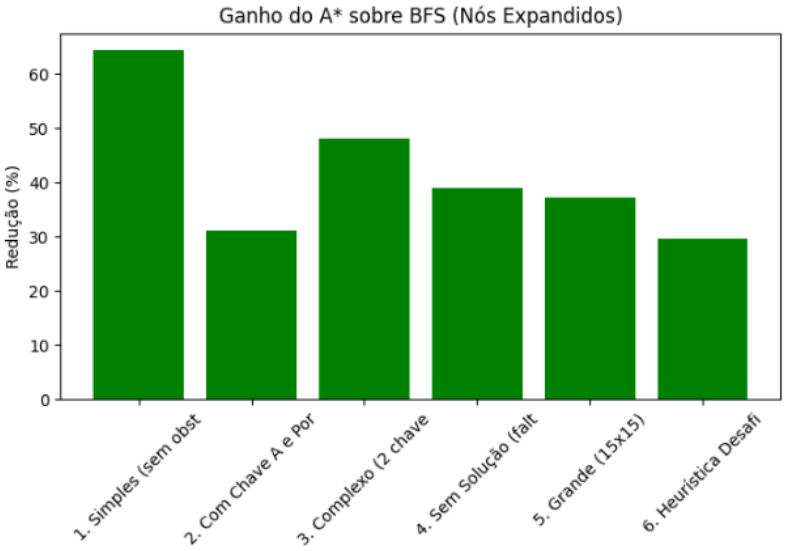

Figura 3. Gráfico do Ganho

O BFS encontrou a solução gerando e expandindo 18 nós, com fator de ramificação médio de 1,00, tempo de aproximadamente 0,0007 segundos e uso de memória de cerca de 0,01 MB, enquanto o A*, utilizando a heurística de *Manhattan*, encontrou o mesmo caminho de 8 movimentos com maior eficiência, gerando 13 nós e expandindo 11, com fator de ramificação médio de 1,18 e tempo de execução de aproximadamente 0,0005 segundos. Apesar de ser admissível e consistente, a heurística evidencia uma limitação do modelo, pois tanto o BFS quanto o A* não consideraram corretamente a restrição da porta sem chave, resultando em uma solução que não seria válida nas regras completas do problema.

No quinto labirinto, também rótulado de grande (15×15), foi utilizado um labirinto 15×15 com diversos obstáculos distribuídos pelo mapa, além de uma chave (K_A) e uma porta (D_A), porém o caminho até o paciente não exigiu o uso da chave, pois existia uma rota livre contornando os obstáculos. O BFS encontrou uma solução ótima com 28 movimentos, onde o robô desce pela primeira coluna até a última linha e depois percorre toda a linha final para a direita até o objetivo. Nesse processo, o algoritmo gerou 367 nós, expandiu 361, apresentou fator de ramificação médio de 1,02, tempo de execução de aproximadamente 0,0217 segundos e uso de memória de cerca de 0,23 MB.

Já o algoritmo A*, utilizando a heurística de *Manhattan*, também encontrou a mesma solução ótima com 28 movimentos e o mesmo caminho do BFS. No entanto, foi mais eficiente em termos de exploração de estados, gerando 255 nós e expandindo 227, com fator de ramificação médio de 1,12. O tempo de execução foi de aproximadamente 0,0149 segundos, um pouco maior que o BFS nesse caso, enquanto o uso de memória foi menor, cerca de 0,15 MB. A Figura 4 apresenta a comparação entre os estados explorados do algoritmos em cada cenário de labirinto, desde o primeiro, caracterizado como um labirinto simples sem obstáculos, até o último, que representa um cenário com heurística desafiadora.

No sexto labirinto, também nomeado de heurística desafiadora, o labirinto é mais desafiador para a heurística porque a chave K_A está em um beco cercado por paredes,

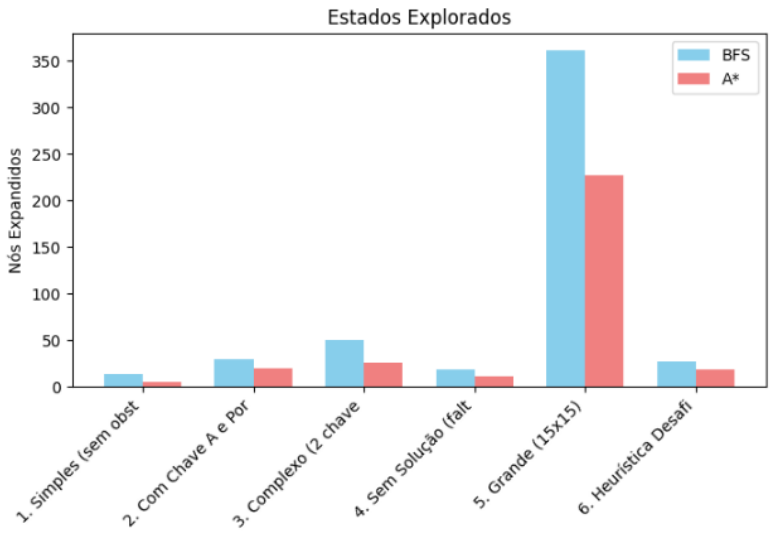

Figura 4. Gráfico dos Estados Explorados

dificultando o acesso direto; o robô precisa coletar a chave antes de atravessar a porta D_A e alcançar o paciente. O BFS encontrou solução ótima com 8 movimentos, indo inicialmente para a direita, depois descendo até a chave e seguindo até a porta e o objetivo. Nesse processo, gerou 31 nós, expandiu 27, teve fator de ramificação médio de 1,15, tempo de execução de aproximadamente 0,0012 segundos e uso de memória de cerca de 0,01 MB.

Já o algoritmo A*, utilizando a heurística de *Manhattan*, também encontrou a mesma solução ótima com 8 movimentos e o mesmo caminho do BFS. No entanto, apresentou maior eficiência na exploração, gerando 22 nós e expandindo 19, com fator de ramificação médio de 1,16. O tempo de execução foi de aproximadamente 0,0009 segundos, e o uso de memória também foi de cerca de 0,01 MB. Esse cenário evidencia uma limitação importante da heurística de Manhattan: embora seja admissível e consistente, ela não considera obstáculos complexos nem a necessidade de coletar chaves, o que pode limitar a escolha de caminhos mais eficientes em situações como essa.

4. Discussão

Para a nível de discussão entre os algoritmos BFS e A* evidencia diferenças importantes em termos de desempenho, eficiência e uso de recursos, ao mesmo tempo em que demonstra que ambos mantêm alta qualidade nas soluções encontradas.

Em relação à qualidade da solução, observa-se que, ambos os algoritmos encontraram soluções ótimas em todos os cenários, resultando no mesmo custo (número de passos) em todos os testes, pois o BFS garante o menor caminho em grafos não ponderados e o A*, com heurística de Manhattan admissível e consistente, também assegura a otimalidade.

No que diz respeito ao número de estados explorados, o A* explora menos estados que o BFS, com redução média de 41,5%, o que melhora o desempenho geral, reduzindo também o tempo de execução em 20,2% e o uso de memória em 25,5%; isso ocorre porque usa heurística para guiar a busca e evitar regiões pouco promissoras, enquanto o

BFS, por ser cego, expande nós uniformemente e tem maior custo computacional.

Entretanto, essa vantagem do A* não é absoluta: no labirinto 15x15, mesmo explorando menos estados e usando menos memória, ele apresentou maior tempo de execução que o BFS devido ao custo adicional do cálculo da heurística e à manutenção de estruturas como filas de prioridade, evidenciando que seu desempenho também depende do *overhead* computacional.

Um ponto central da análise é a sensibilidade à heurística: o desempenho do A* depende diretamente da qualidade da heurística utilizada. A heurística de *Manhattan* mostrou-se eficaz na maioria dos cenários, reduzindo significativamente o número de estados explorados, especialmente em ambientes mais simples. No entanto, apresenta limitações por não considerar obstáculos, portas ou a necessidade de coleta de chaves, o que pode levar à subestimação do custo real e reduzir a eficiência do algoritmo em cenários mais complexos. Assim, quanto mais informativa e próxima do custo real for a heurística, melhor tende a ser o desempenho do A*.

Por fim, em relação à convergência para ótimos locais, ambos os algoritmos são robustos: o BFS, por ser exaustivo, sempre encontra a solução ótima global quando ela existe, enquanto o A*, ao utilizar uma heurística admissível e consistente, também evita ótimos locais e mantém garantias de completude e otimalidade, sendo assim ambos confiáveis nesse aspecto.

5. Conclusão

Concluir-se que, na comparação entre BFS e A*, o A* se mostrou mais eficiente na maioria dos cenários, reduzindo estados explorados, tempo e memória sem perder otimalidade, mas seu desempenho depende da qualidade da heurística, que pode ser limitada em casos complexos. Ambos são completos e ótimos, porém a validação correta das restrições é essencial para garantir soluções válidas.